
✓ Basic SQL Questions with Answers

1. What is SQL?

- *Structured Query Language used to communicate with databases.*

2. What are the different types of SQL commands?

- *DDL (CREATE, DROP), DML (INSERT, UPDATE), DCL (GRANT, REVOKE), TCL (COMMIT, ROLLBACK), DQL (SELECT).*

3. What is the difference between `WHERE` and `HAVING` ?

- *`WHERE` filters rows before grouping, `HAVING` filters after grouping.*

4. How do you retrieve all columns from a table?

```
SELECT * FROM table_name;
```

5. How do you rename a table?

```
ALTER TABLE old_name RENAME TO new_name;
```

6. How do you add a column to a table?

```
ALTER TABLE table_name ADD column_name datatype;
```

7. What is a primary key?

- *A unique, non-null column or combination of columns that uniquely identifies each row.*

8. What is a foreign key?

- *A column that creates a relationship between two tables.*

9. How do you retrieve unique values from a column?

```
SELECT DISTINCT column_name FROM table_name;
```

10. What is the difference between `NULL` and 0?

- `NULL` means no value; 0 is an actual number.

11. How do you sort results?

```
SELECT * FROM table_name ORDER BY column_name ASC|DESC;
```

12. How do you filter results based on a condition?

```
SELECT * FROM table_name WHERE condition;
```

13. How do you update data in a table?

```
UPDATE table_name SET column1 = value1 WHERE condition;
```

14. How do you delete rows from a table?

```
DELETE FROM table_name WHERE condition;
```

15. What is the difference between `INNER JOIN` and `LEFT JOIN` ?

- `INNER JOIN` returns matching rows; `LEFT JOIN` returns all from left and matched from right.

16. How do you count rows in a table?

```
SELECT COUNT(*) FROM table_name;
```

17. What does `GROUP BY` do?

- Groups rows that have the same values into summary rows.

18. How do you get the maximum value from a column?

```
SELECT MAX(column_name) FROM table_name;
```

19. What is normalization?

- Process of organizing data to reduce redundancy.

20. What is the default sort order in SQL?

- *Ascending (ASC).*

Medium SQL Questions with Answers

1. Get second highest salary from employee table

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

2. List departments with more than 5 employees

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

3. Find duplicate rows in a table

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING COUNT(*) > 1;
```

4. What is a subquery?

- *A query inside another query, used in WHERE, FROM or SELECT clause.*

5. What is a correlated subquery?

- *A subquery that uses columns from the outer query.*

6. What is a CASE statement?

```
SELECT name,
CASE
    WHEN marks >= 90 THEN 'A'
    WHEN marks >= 75 THEN 'B'
    ELSE 'C'
```

```
END AS grade
FROM students;
```

7. Get employees who joined in the last 30 days

```
SELECT * FROM employees
WHERE join_date >= CURRENT_DATE - INTERVAL '30 days';
```

8. Difference between UNION and UNION ALL ?

- UNION removes duplicates, UNION ALL does not.

9. Delete duplicate rows keeping only one

```
DELETE FROM employees
WHERE id NOT IN (
    SELECT MIN(id) FROM employees GROUP BY email
);
```

10. Find the Nth highest salary (e.g., 3rd highest)

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

11. List customers who didn't place any orders

```
SELECT * FROM customers
WHERE customer_id NOT IN (
    SELECT customer_id FROM orders
);
```

12. Difference between RANK() and DENSE_RANK()

- RANK() skips numbers on ties; DENSE_RANK() does not.

13. How do you create an index?

```
CREATE INDEX idx_name ON table_name(column_name);
```

14. Update values using JOIN

```
UPDATE A
SET A.value = B.value
FROM B
WHERE A.id = B.id;
```

15. What is a window function?

- Functions like `ROW_NUMBER()` , `RANK()` applied over partitions using `OVER()` clause.



Advanced SQL Questions with Answers

1. Get cumulative salary per department

```
SELECT department_id, employee_id, salary,
       SUM(salary) OVER(PARTITION BY department_id ORDER BY employee_id) AS cumulative_salary
FROM employees;
```

2. Find employees with salaries above department average

```
SELECT *
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);
```

3. Pivot rows to columns (PostgreSQL)

```
SELECT *
FROM crosstab(
    'SELECT emp_id, year, salary FROM salary_data ORDER BY 1,2',
    'SELECT DISTINCT year FROM salary_data ORDER BY 1'
) AS ct(emp_id INT, y2022 INT, y2023 INT);
```

4. Recursive query to get hierarchy

```

WITH RECURSIVE emp_cte AS (
  SELECT id, name, manager_id FROM employees WHERE manager_id IS NULL
  UNION
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  INNER JOIN emp_cte cte ON e.manager_id = cte.id
)
SELECT * FROM emp_cte;

```

5. Find gaps in a sequence

```

SELECT curr.id + 1 AS missing_id
FROM employees curr
LEFT JOIN employees next ON curr.id + 1 = next.id
WHERE next.id IS NULL;

```

6. Top 2 salaries per department

```

SELECT * FROM (
  SELECT *, RANK() OVER(PARTITION BY department_id ORDER BY salary DESC) as rnk
  FROM employees
) t
WHERE rnk <= 2;

```

7. Delete all except latest N records

```

DELETE FROM logs
WHERE id NOT IN (
  SELECT id FROM logs ORDER BY timestamp DESC LIMIT 100
);

```

8. Find running average of last 3 orders per customer

```

SELECT customer_id, order_id, AVG(amount) OVER(
  PARTITION BY customer_id ORDER BY order_date ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS moving_avg
FROM orders;

```

9. JSON query example (PostgreSQL)

```
SELECT json_data->>'name' AS name
FROM users
WHERE json_data->>'active' = 'true';
```

10. Find overlapping time ranges

```
SELECT a.*, b.*
FROM bookings a, bookings b
WHERE a.id <> b.id
      AND a.start_time < b.end_time
      AND a.end_time > b.start_time;
```
